Data Augmentation in Property Graphs using agentic AI: measuring human effort, task difficulty, environmental footprint and accuracy[⋆,⋆⋆]

Felipe Vasconcelos[a,*], Cristina Aguiar[a], Alexandre Chanson[b], Mirian Halfeld-Ferrari[c], Patrick Marcel[c] and Verónika Peralta[b]

[a] *ICMC, University of São Paulo, São Carlos, Brazil*
[b] *LIFAT, Université de Tours, France*
[c] *Université d'Orléans, INSA CVL, LIFO, UR 4022, Orléans, France*

ARTICLE INFO

Keywords:
TO BE CHANGED
Data augmentation
property graphs
LLMs

ABSTRACT

TO BE CHANGED

This paper investigates the use of large language models (LLMs) to complete missing attribute values in property graphs. Addressing this problem requires leveraging both the structural context of a node and the attribute information already present in the graph, and we explore the use of LLMs to infer or retrieve plausible attribute values from external sources. We propose an experimental protocol to evaluate LLM-based data augmentation in terms of accuracy, prompting effort, and environmental impact, and introduce a difficulty metric inspired by perplexity. Our preliminary results show that most of the models tested struggle with augmentation tasks and that context is useful for difficult tasks, with variations in the model and prompting strategy used.

A general idea of the structure of the paper[Mirian]

1. Introduction Below a first version[Mirian]
2. Methodology (Pipeline) I am working on this part. I started thinking in describing the whole architecture of each method : like in the slides of Felipe. But, at the end, I am putting the description of the unified protocol here... to describe the 2 sub-routines. [Mirian]
3. Experimental Protocol To be adjusted to the idea we have discussed at the last meeting.
   ON October 5th: I am inserting the description of the protocols in the previous section[Mirian]
4. Place the metrics in a separated section
5. Experiments
6. Related work see, perhaps after introduction[Mirian]
7. Conclusion

⋆This work was partially supported by JUNON Program (DATA/LIFO) with financial support of Région Centre-Val de Loire, France, and partially supported by project TopOL with financial support of the French National Research Agency (ANR).

⋆⋆The first and second authors were supported by the São Paulo Research Foundation (FAPESP), the Brazilian Federal Research Agency (CNPq), and the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior, Brasil (CAPES), Finance Code 001.

*Corresponding author

felipevsc@usp.br (F. Vasconcelos); cdac@icmc.usp.br (C. Aguiar); Alexandre.Chanson@univ-tours.fr (A. Chanson); mirian@univ-orleans.fr (M. Halfeld-Ferrari); Patrick.Marcel@univ-orleans.fr (P. Marcel); Veronika.Peralta@univ-tours.fr (V. Peralta)

ORCID(s): 0009-0007-7922-9746 (F. Vasconcelos); 0000-0002-7618-1405 (C. Aguiar); 0000-0001-9195-5950 (A. Chanson); 0000-0003-2601-3224 (M. Halfeld-Ferrari); 0000-0003-3171-1174 (P. Marcel); 0000-0002-9236-9088 (V. Peralta)

1. Introduction

Graph databases have become an established paradigm for modeling complex, highly interconnected domain knowledge. Because real-world graphs are frequently constructed by integrating heterogeneous and asynchronous sources, they naturally evolve over time through the progressive addition of nodes, relationships, and metadata attributes. Consequently, many real-world property graphs suffer from data incompleteness—manifesting as missing attribute values, sparse properties, or undefined relationships. This incompleteness substantially degrades the performance of downstream graph querying, analytical tasks, and reasoning algorithms. To mitigate graph sparsity without relying on manual curation, automated techniques for data generation, discovery, and attribute augmentation have emerged as promising mechanisms for graph completion [2].

While existing research has mainly focused on inferring or predicting missing edges, the completion of node attribute values remains underexplored. Addressing this problem requires considering both the structural context of a node and the attribute information already available in the graph. In this paper, we assume a scenario in which, for each node, we construct prompts that include: (1) a set G of existing ("property", "value") pairs, and (2) a set H of property names for which values are sought. The selection of properties included in G and H takes into account the popularity of nodes and attributes within the graph, allowing us to analyze different task difficulties.

In our previous work [**?**], to solve this problem, we proposed an approach for completing data graphs with missing values for selected node attributes, exploring the use of Large Language Models (LLMs) to infer or retrieve plausible attribute values. This initial approach relied on a static input-to-output pipeline (Input → Prompt/Model → Output), leveraging the model's internal parametric knowledge alongside the graph-based notion of context introduced in [**?**]. While this static method effectively generates high-confidence candidate values for missing attributes, enforcing strict confidence prompts can lead to conservative model behavior—resulting in abstention and, consequently, lower recall.

As an alternative execution pathway aimed at mitigating the abstention rates of static pipelines, we introduce an agentic AI architecture operating as a dedicated *Search Agent*.

Thus, in this paper, we propose a **unified Graph Augmentation Framework** for Property Graph completion. Rather than viewing static prompting and external agentic search in isolation, our framework formalizes two distinct operational pathways:

1. **Strategy A (Static Parametric Pathway):** A pipeline relying strictly on the LLM's internal parametric memory and guided by contextual information [**?**].

2. **Strategy B (Single-Turn Agentic Pathway):** A lightweight, single-interaction Search Agent that combines the LLM's reasoning engine with an external retrieval tool (e.g., Tavily). Given known (attribute, value) pairs in G and target attributes H, the agent plans a retrieval strategy, decomposes intent into parallel or sequential sub-queries, executes a single-turn tool invocation, and synthesizes raw evidence into graph updates without introducing complex, multi-turn conversational overhead.

Through a rigorous empirical study, we evaluate and compare **Strategy A** and **Strategy B** across key performance dimensions.

Main contributions.

- **A unified Graph Augmentation Framework** that structures Property Graph attribute completion into static or agentic operational modes through a general execution protocol, specialized according to the underlying search model used.
- **A lightweight, single-turn Search Agent (Strategy B)** capable of parallel query decomposition and sequential dependency resolution using an external web search tool (e.g., Tavily).
- **An new automated task construction algorithm** that generates structured tuples (n, G, H) of known context properties G and target missing attributes H for each node n in the property graph.
- **A thorough comparative analysis** between Strategy A [**?**] and Strategy B, evaluating performance across accuracy, prompting effort, and execution sustainability while refining the evaluation metrics from [**?**] for task difficulty and computational effort.
- **An expanded empirical benchmark** spanning three distinct property graph datasets, significantly broadening the single-dataset evaluation scope established in [**?**].

Paper organization. TO BE DONE

Algorithm 1 Unified Evaluation Protocol

Require: Set of models M, set of nodes from property graph PG of a given type t

Ensure: Evaluation results *Result* mapping tasks and strategies to performance metrics (accuracy, effort, sustainability)

// Phase 1: Structural Analysis & Task Definition

1: **for** each node $n \in N$ **do**
2: Compute node centrality $cent(n)$ and local context $context(n)$
3: **for** each property p found in $context(n)$ **do**
4: Compute the popularity $pop(p)$ of p
5: **end for**
6: **end for**
7: $TASKS \leftarrow taskDefinition(N)$ ▷ Generates tuples (n, G_n, H_n); see Algorithm 4

// Phase 2: Dual-Pathway Execution & Scoring

8: $Result \leftarrow \emptyset$
9: **for** each model $m \in M$ **do**
10: **for** each task $(n, G_n, H_n) \in TASKS$ **do**
11: $R_{\text{StratA}} \leftarrow SimpleLLMRoutine(m, n, G_n, H_n, r)$ ▷ Strategy A (Parametric)
12: ▷ Returns results across all templates T
13: $R_{\text{StratB}} \leftarrow AgenticAIRoutine(m, n, G_n, H_n)$ ▷ Strategy B (Agentic Search)
14: **for** strategy $\sigma \in \{\text{StratA}, \text{StratB}\}$ **do**
15: $score \leftarrow \big((n, G_n, H_n), m, \sigma, \text{Acc}(R_\sigma), \text{Effort}(R_\sigma), \text{Sust}(R_\sigma)\big)$
16: $Result \leftarrow Result \cup \{score\}$
17: **end for**
18: **end for**
19: **end for**
20: **return** *Result*

2. The Unified Graph Augmentation Framework: overview

Put here the general protocol??? I am writing this first version, imagining we refer to it... I am trying a new version of the general protocol. TO CHECK[Mirian]

We present a unified Graph Augmentation Framework governed by Algorithm 1. The evaluation protocol operates in two distinct phases: (i) structural analysis and task construction, where node centralities $cent(n)$ and property popularities $pop(p)$ are computed to sample benchmark tasks $TASKS$ (Lines 1 - 6), and (ii) execution and comparative scoring (from line 7). The protocol is strategy-agnostic: for each task (n, G_n, H_n), it dispatches the input to two distinct operational pathways—*SimpleLLMRoutine* (Strategy A) and *AgenticAIRoutine* (Strategy B)—measuring accuracy, prompting effort, and execution sustainability under identical graph conditions.

2.1. Strategy A: Static Parametric Pathway

Strategy A implements the static, memory-bound methodology introduced in [**?**], as illustrated in Figure **??**Figure from Felipe's slides[Mirian]. In this approach, we leverage the internal parametric knowledge of Large Language Models (LLMs) grounded in known graph context to infer or complete missing attribute values. The execution flow is formalized in the *SimpleLLMRoutine* procedure (see line 11 of Algorithm 1 and Algorithm 2).

Given a task (n, G_n, H_n), Algorithm 2 builds a zero-shot prompt template. The request is dispatched directly to model m, relying strictly on its parametric memory without invoking external APIs. To account for generation variance across non-deterministic LLM runs, each prompt template is executed multiple times (Algorithm 2, lines 5–8). Finally, the resulting candidate completions are aggregated across runs to produce the final attribute assignments (Line 9). To account for generation non-determinism across independent sampling runs ($r \in \mathbb{N}_{\text{odd}}$), the *Aggregate* function applies majority voting over extracted attribute values. For a given task (n, G_n, H_n) and template t, the value appearing in the majority of r runs is retained as $R_{m,e}$. Is this correct??[Mirian]

Algorithm 2 SimpleLLMRoutine (Strategy A)

Require: Model m, task (n, G_n, H_n), set of prompt templates T, number of run iterations $r \in \mathbb{N}_{\text{odd}}$
Ensure: Set $TemplateResults$ containing candidate completions aggregated across r runs for each template $t \in T$

$TemplateResults \leftarrow \emptyset$
for each prompt template $t \in T$ **do**
  $e \leftarrow Instantiate(t, n, G_n, H_n)$ ▷ Inject task into template t
  $Runs \leftarrow \emptyset$
  for $k \leftarrow 1$ *to* r **do** ▷ Sampling across r runs
    $res \leftarrow PromptModel(m, e)$ ▷ Zero-shot parametric inference
    $Runs \leftarrow Runs \cup \{res\}$
  end for
  $R_{m,e} \leftarrow Aggregate(Runs, m, e)$ ▷ PUT A COMMENT HERE. Majority voting ??[Mirian]
  $TemplateResults \leftarrow TemplateResults \cup \{(t, R_{m,e})\}$
end for
return $TemplateResults$

3. Experimental protocol

This section presents our experimental protocol, detailed in Algorithm 3. The aim is to evaluate three metrics representing the prompting effort, result accuracy and process sustainability when performing entity augmentation tasks of varying difficulty in a property graph.

Algorithm 3 Protocol

Require: A set M of models, a set T of prompt templates, an odd integer r for the number of runs, a set of node N from property graph PG of a given type t
Ensure: Accuracy, effort, sustainability for combinations of models in M, instances of template in T

for each node n of N **do**
  compute $cent(n)$ the centrality of n and $context(n)$ the context of n
  for each property p in $context(n)$ **do**
    compute $pop(p)$ the popularity of p
  end for
end for
$TASKS = taskDefinition(N)$ ▷ See Algorithm 4
$Result = \emptyset$
for each model m in M **do**
  for each prompt template t in T **do**
    for each task $(n, H, G) \in TASKS$ **do**
      let e be the edited prompt t with n, H, G
      $R = \emptyset$
      for 1 to r **do**
        let res be the result of prompting model m with e
        $R = R \cup \{res\}$
      end for
      final results $R_{m,e} = aggregate(R, m, e)$
      $scores = ((n, H, G), Accuracy(R_{m,e}), effort(R_{m,e}), sustainability(R_{m,e}))$
      $Result = Result \cup \{scores\}$
    end for
  end for
end for
return Result

Algorithm 4 Tasks definition

Require: A set of nodes N with their respective contexts and centralities ; integers max_n, max_H, max_G
Ensure: A set of $27 \times max_n \times max_H \times max_G$ tasks varying in terms of Difficulty

$TASKS = \emptyset$; $D = \emptyset$; $P = \emptyset$
for all elements e in $N \cup context(N)$ **do**
  for all properties p of e **do**
    $P = P \cup (p, pop(p) \times cent^p(e))$
  end for
end for
Let P_0, P_1, P_2, P_3, P_4 be the quartiles of $pop(p) \times cent^p(e)$ for P
Let Q_0, Q_1, Q_2, Q_3, Q_4 be the quartiles of $cent()$ for N
for $Q_i \in \{Q_1, Q_2 \cup Q_3, Q_4\}$ **do**
  $NODES =$ draw max_n node n such that $cent(n) \in [Q_{i-1}, Q_i]$
  for $n \in NODES$ **do**
    for $size_H \in [1, max_H]$ **do**
      for $size_G \in [1, max_G]$ **do**
        for $P_i \in \{P_1, P_2 \cup P_3, P_4\}$ **do**
          $H =$ draw $size_H$ properties p of n such that $pop(p) \times cent^p(n) \in [P_{i-1}, P_i]$
          for $P_i \in \{P_1, P_2 \cup P_3, P_4\}$ **do**
            $G =$ draw $size_G$ properties p of $(n \cup context(n)) - H$ such that $pop(p) \times cent^p(e) \in [P_{i-1}, P_i]$
            $TASKS = TASKS \cup (n, H, G)$
          end for
        end for
      end for
    end for
  end for
end for
return $TASKS$

3.1. Metrics

Algorithm 3 uses three metrics: accuracy, prompt formulation effort and sustainability. Accuracy consists of comparing the result given by the model with the ground truth, i.e., the values of properties hidden. Both results and ground truth are sets of pair p, v with p a property name (or property for short) and v a value. Measuring accuracy is done with the classical F1 score, aggregating precision and recall. Prompt Formulation Effort (PFE) measures human effort in crafting the different prompts passed to the model. As explained in Section **??** we have compact and base templates, and can instantiate these templates using 0 shot, few shots and Chain Of Thought (CoT). A simple proxy metric for this effort is the number of tokens in the prompt. Finally, sustainability relates to the choice of model and the parameters used. We measure this with a composite score described in details in Section **??**.

3.2. Entity augmentation tasks and their difficulty

The task of the model is to predict the values of some node's properties, hidden from it. Formally, a task is a tuple (n, H, G) where n is a node, H is the set of properties for which we seek to predict values and G is the set of properties with their values, known by the model. Tasks are constructed by Algorithm 4. We posit that the difficulty of the task will vary with both the characteristics of the properties to predict, i.e., H, and the information passed to the model, i.e., G, that include the specific node n, some of its properties and some properties in its context.

should we also account for "domain" difficulty? But not sure how to measure it.Patrick

Precisely, given a task (n, H, G), we assume the following:

1. the more properties in H and the less in G (including properties from the node's context), the more difficult the task,

2. the more central n, the easier its properties in H will be to predict, and the less difficult the task. For instance, if airports are ranked by node centrality (using e.g., page rank), then airports of high centrality likely correspond to big hubs whose properties should be easier to retrieve by the model,

3. the more popular the property in H (in terms of presence in the graph, i.e., the probability that the property has a value when observing a node of a given type), the easier to predict and the less difficult the task,

4. the higher the centrality of a property's element (node or arc) of n's context in G, the easier it is for the model to use it and the less difficult the task (e.g., if the airport city name of some highly central cities is in G, it will be easier for the model to retrieve the airport's hidden properties),

5. the more popular the property in G, the more likely it can appear in the training examples of the model, and consequently the easier for the model to use it, and the less difficult the task.

To account for the respective sizes of H and G (1), we use the probability of drawing a property p of H from $H \cup G$, i.e., $P(p \in H) = \frac{|H|}{|H|+|G|}$. Note that $P(p \in H) = 0$ when $H = \emptyset$ since there is nothing to predict, while $P(p \in H) = 1$ when $G = \emptyset$ since nothing is given to help the prediction. The case $H = G = \emptyset$ is undefined.

Difficulty must also account for the *centrality* of n (2) and the elements in its *context* (4).

Definition (Centrality of an element) The popularity of an element (node or edge) e is given by a centrality measure $cent(e)$, assumed normalized, for this element. Alternatively, centrality can be replaced with, when randomly drawing a node in the graph, the probability of the node being source or target of the relationship.

To account for n's centrality, we use the surprise brought by its centrality, e.g., $-log(cent(n))$. Difficulty of a task should also account for the centrality of the elements in the context of n whenever there are properties of the context of n in G.

Definition (Context) The context of a node n is a set of properties coming from n, its direct neighbor nodes, the relationships to its direct neighbor nodes, and the properties along paths where relationships have a $n : 1$ cardinality, starting from n (like e.g., relationship IN between a city and a country since a city is IN only one country). A formal definition of context is given in [1]. It is computed with function $context()$.

Finally, to account for 3 and 4, we will use the surprise brought by the popularity of properties in H and G respectively, weighted by the centrality of their element (node or edge) of origin.

Definition (Popularity of a property) Given a property p of an element (node or edge) e of type t, we call $pop(p)$ the popularity of p, defined as the probability of observing a (non null) value for property p when drawing an element in the set of elements of type t. This gives a probability distribution P for random variable p with two outcomes: $P(p \neq null)$ and $P(p = null)$. Note that we do not consider the probability distribution over the values of properties because the majority of values will be specific to the element and be present only once (e.g., latitude, name, length of runways, etc.).

Definition (Difficulty of a task) Let $T = (n, H, G)$ be a task. $cent(n)$ is the centrality of n, assumed normalized. Given a property p, $pop(p)$ is the popularity of p, i.e., the probability of appearance of p among the set of all properties. $cent^p(e)$ is the centrality of the element e the property p comes from. The difficulty of task T is the geometric mean of the probability of seeing a property from H, the surprise brought by the centrality of n and the surprise brought by the popularity (weighted by centrality of their origin) of the properties in H and G:

$$\text{Difficulty}(T) = \sqrt[3]{\frac{|H|}{|H|+|G|} \times -log(cent(n)) \times \sum_{p \in H \cup G} -log((pop(p) \times cent^p(e)))}$$

Note: I changed a bit the formula agreed with Veronika, which was: $\frac{|H|}{|H|+|G|} \times -log(cent(n)) \times -log(\sum_{p \in H \cup G}(pop(p) \times cent^p(e)))$. First I used the geometric mean instead of just the product, then I moved the sum out of the last log since AFAIK surprise is defined by -log of some probability.Patrick

Element		Property	
e	$cent(e)$	p	$pop(p)$
e_1	0.7	a	0.6
e_1	0.7	b	0.4
e_2	0.4	c	0.1
e_2	0.4	d	0.3
e_2	0.4	e	0.6
e_3	0.2	f	0.2
e_3	0.2	g	0.1
e_3	0.2	h	0.2

Node	Task H and G	Intuitive difficulty	$\frac{\|H\|}{\|H\|+\|G\|}$	$-log(cent(n))$	$\sum_{p\in H\cup G}$	Difficulty
e_1	H1 = $\{b\}$ G1 = $\{a,d\}$	easy	0.33	0.51	6.14	1.01
e_2	H2 = $\{e\}$ G2 =$\{f,h\}$	medium easy	0.33	1.32	11.34	1.71
e_2	H3 = $\{c,d\}$ G3 = $\{b\}$	medium hard	0.66	1.32	9.54	2.03
e_3	H4 = $\{f,h\}$ G4 = $\{g\}$	hard	0.66	2.32	14.93	2.84

Table 1
Example of various tasks and their difficulty.

Example Table 1 illustrates the difficulty of various tasks. Table 1 (left) lists various properties p coming from different elements e, with the centrality of e and the popularity of p. Table 1 (right) lists various tasks built with these properties, and details the computation of their difficulty. It can be observed that, as desired, tasks with a central node and popular properties, like (e_1, H_1, G_1), have a low difficulty while tasks with a node of lower centrality and less popular properties, like $(e_3, H4, G4)$, have a much higher difficulty. Desirably, medium easy tasks like $(e_2, H2, G2)$, where H has popular (easy to predict) properties and G has less popular properties, are scored less difficult than tasks like $(e_2, H3, G3)$ where H has unpopular properties even if G has popular ones.
Important: is this desirable?Patrick Note that, two tasks $T_1 = (n_1, X, Y), T_2 = (n_2, Y, X)$, i.e., where the hidden of T_1 is the given of T_2 and vice versa, such that $|X| = |Y|$ and $cent(n_1) = cent(n_2)$, have the same difficulty.

3.3. Overview of the protocol

Algorithm 3 starts by computing nodes centralities, contexts and properties popularities (lines 1-7). Based on those, tasks are defined by drawing randomly (i) nodes of various centralities and for each (ii) properties of various popularities in sets H and G. (see Algorithm 4). Then for each model, tasks are used to instantiate various prompt templates (line 10-13) and models are prompted (line 16). Finally, as the model's answers to a specific prompt may vary, we will pass the prompt run many times (lines 15-18), and aggregate (line 19) the model results (see Section **??** for details).

4. TO DISCUSS

We have discussed: Cristina, Felipe and me... on how to present the paper. This is a first proposal to discuss tomorrowMirian.

The idea is to present both approaches as sub-routines of a global protocol. I am trying to express this in the following algorithm

References

[1] C. Aguiar, J. Chabin, A. Chanson, M. Halfeld-Ferrari, N. Hiot, N. Labroche, P. Marcel, V. Peralta, and F. Vasconcelos. Extracting node comparison insights for the interactive exploration of property graphs, 2025.

[2] J. Zhou, C. Xie, S. Gong, et al. Data augmentation on graphs: A technical survey. *ACM Comput. Surv.*, 57(11):274:1–274:34, 2025.

Algorithm 5 Protocol

Require: A set M of models, a set of node N from property graph PG of a given type t
Ensure: Accuracy, effort, sustainability for combinations of models in M

for each node n of N **do**
  compute $cent(n)$ the centrality of n and $context(n)$ the context of n
  for each property p in $context(n)$ **do**
    compute $pop(p)$ the popularity of p
  end for
end for
$TASKS = taskDefinition(N)$ ▷ See Algorithm 4
$Result = \emptyset$
for each model m in M **do**
  for each task $(n, H, G) \in TASKS$ **do**
    compute results $R_{simpleLLM}$ with $SimpleLLMRoutine$
    compute results $R_{AgenticAI}$ with $AgenticAIRoutine$
  end for
  $scores = ((n, H, G), Accuracy(R_{m,e}), effort(R_{m,e}), sustainability(R_{m,e}))$
  $scores = ((n, H, G), Accuracy(R_{AgenticAI}), effort(R_{AgenticAI}), sustainability(R_{AgenticAI}))$
  $Result = Result \cup \{scores\}$
end for
return Result

Algorithm 6 SimpleLLMRoutine

Require: The model m, the task $task = (n, H, G)$, a set of prompt templates T an odd integer r for the number of runs,
Ensure: The best results obtained for $task$ when using simple LLM strategy and the prompt template used to obtain it

for each prompt template t in T **do**
  let e be the edited prompt t with n, H, G
  $R = \emptyset$
  for 1 to r **do**
    let res be the result of prompting model m with e
    $R = R \cup \{res\}$
  end for
  final results $R_{m,e} = aggregate(R, m, e)$
end forreturn $R_{m,e}$

Algorithm 7 AgenticAI

Require: The model m, the task $task = (n, H, G)$, an odd integer r for the number of runs,
Ensure: The best results obtained for $task$ when using simple Agentic AI strategy

$R = \emptyset$
for 1 to r **do**
  let res be the result given by the Agentic-AI for model m
  $R = R \cup \{res\}$
end for
final results $R_m = aggregate(R, m)$
return R_m